

Program 1 & 3: Install Python libraries and scikit-learn

```
!pip install numpy pandas scikit-learn matplotlib seaborn openpyxl
```

Program 2: Introduce and set up the libraries

```
import numpy as np
```

```
import pandas as pd
```

```
import sklearn
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
print("All libraries installed and scikit-learn is ready!")
```

Pr-4

```
import pandas as pd
```

Creating quick dummy files so you don't get a FileNotFoundError in the exam

```
pd.DataFrame({'Name': ['Jack', 'Tony'], 'Age': [22, 45]}).to_csv('data.csv', index=False)
```

```
pd.DataFrame({'City': ['Bangalore', 'New York'], 'Score': [95, 88]}).to_excel('data.xlsx', index=False)
```

1. Load and explore CSV

```
df_csv = pd.read_csv('data.csv')
```

```
print("--- CSV Data ---")
```

```
print(df_csv.head())
```

2. Load and explore Excel

```
df_excel = pd.read_excel('data.xlsx')
```

```
print("\n--- Excel Data ---")
```

```
print(df_excel.head())
```

Pro-5

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Using a simple dataset
```

```
data = {'Student': ['A', 'B', 'C', 'D'], 'Hours_Studied': [2, 4, 6, 8], 'Marks': [40, 65, 80, 95]}
```

```
df = pd.DataFrame(data)
```

```
# Scatter Plot
```

```
plt.figure(figsize=(10, 4))
```

```
plt.subplot(1, 2, 1)
```

```
plt.scatter(df['Hours_Studied'], df['Marks'], color='blue')
```

```
plt.title('Scatter Plot: Hours vs Marks')
```

```
plt.xlabel('Hours Studied')
```

```
plt.ylabel('Marks')
```

```
# Bar Chart
```

```
plt.subplot(1, 2, 2)
```

```
plt.bar(df['Student'], df['Marks'], color='orange')
```

```
plt.title('Bar Chart: Student Marks')
```

```
plt.xlabel('Student')
```

```
plt.ylabel('Marks')
```

```
plt.tight_layout()
```

```
plt.show()
```

Pr-6

```
import pandas as pd

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import LabelEncoder, StandardScaler

# Dummy data with missing values and categories

df = pd.DataFrame({

    'Age': [25, 30, None, 35],

    'City': ['Bangalore', 'Mumbai', 'Bangalore', 'Delhi'],

    'Salary': [50000, 60000, 55000, None]

})

print("Original Data:\n", df, "\n")


# 1. Handle Missing Data (Fill with mean)

imputer = SimpleImputer(strategy='mean')

df[['Age', 'Salary']] = imputer.fit_transform(df[['Age', 'Salary']])


# 2. Encode Categorical Variables (Convert City to numbers)

le = LabelEncoder()

df['City'] = le.fit_transform(df['City'])


# 3. Feature Scaling (Standardize Age and Salary)

scaler = StandardScaler()

df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])

print("Processed Data:\n", df)
```

Pr-7

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score


# Load data

iris = load_iris()

X, y = iris.data, iris.target


# Split into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Train the k-NN Classifier

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)


# Evaluate performance

predictions = knn.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

print(f"k-NN Model Accuracy: {accuracy * 100}%")
```

Pr-8

```
from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

from sklearn.datasets import make_regression
```

```
import matplotlib.pyplot as plt
```

```
# Generate continuous target variable data
```

```
X, y = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Train the model
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
# Visualize the regression line
```

```
predictions = model.predict(X_test)
```

```
plt.scatter(X_test, y_test, color='blue', label='Actual Data')
```

```
plt.plot(X_test, predictions, color='red', label='Regression Line')
```

```
plt.title("Linear Regression")
```

```
plt.legend()
```

```
plt.show()
```

Pr-9

```
from sklearn.datasets import load_iris
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
import matplotlib.pyplot as plt
```

```
# Load data
```

```
iris = load_iris()
```

```
X, y = iris.data, iris.target
```

```
# Train the Decision Tree
```

```
tree_model = DecisionTreeClassifier(max_depth=3, random_state=42)
```

```
tree_model.fit(X, y)
```

```
# Visualize the splits
```

```
plt.figure(figsize=(12, 8))
```

```
plot_tree(tree_model, feature_names=iris.feature_names, class_names=iris.target_names,  
filled=True)
```

```
plt.title("Decision Tree Visualization")
```

```
plt.show()
```

Pr-10

```
from sklearn.cluster import KMeans
```

```
from sklearn.datasets import make_blobs
```

```
import matplotlib.pyplot as plt
```

```
# Generate random cluster data (simulating the online retail problem)
```

```
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=42)
```

```
# Train K-Means
```

```
kmeans = KMeans(n_clusters=4, random_state=42)
```

```
kmeans.fit(X)
```

```
# Visualize Clusters
```

```
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_, cmap='viridis', s=50, alpha=0.8)
```

```
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', s=200, marker='X',  
label='Centroids')
```

```
plt.title("K-Means Clustering")
```

```
plt.legend()
```

```
plt.show()
```